Synopsis

TCS 203 : Fundamentals of Artificial Intelligence and Machine Learning

Project Based Learning : Phase 1

Campus Wifi Network Congestion Predictor

Problem Id: 25

Team Leader

Parth Raj Jaiswal

2028792 | Section K

Team Member 2

Prrajval Tadiyal

2028811 | Section K

Team Member 3

Bhavneet Singh

2028691 | Section K

Problem Description

Campus Wi-Fi networks serve thousands of students, faculty, and staff simultaneously, making network congestion a persistent and disruptive challenge. When too many devices connect at once, or when bandwidth demand exceeds capacity, users experience slow speeds, high latency, dropped connections, and poor overall network performance — directly impacting academic activities like online classes, research, and digital examinations.

The core problem is the inability to proactively detect and predict congestion before it severely degrades network quality. Traditional network monitoring is reactive — administrators only respond after performance has already collapsed. This results in poor user experience and inefficient resource management.

This project addresses that gap by developing an end-to-end machine learning classification system that analyzes real-time network performance data — including number of connected devices, bandwidth usage, network latency, packet loss rate, and access point load — to automatically predict the congestion level as Low, Medium, or High.

In real life, such a system can be deployed by university IT departments to monitor network health continuously, anticipate peak load periods, redistribute traffic intelligently across access points, and make data-driven infrastructure decisions. Ultimately, it shifts network management from a reactive model to a proactive, intelligent one — improving connectivity reliability for the entire campus community.

Steps of Implementation

Fig. 1 ML pipeline architecture

Phase 1 **: Data Collection**

The dataset used in this project was synthetically generated to simulate real campus Wi-Fi network behaviour. It consists of 5,575 records and 15 features including number of connected devices, bandwidth usage, AP load, network latency, packet loss rate, signal strength, retransmission rate, throughput, active sessions, channel utilization, timestamp hour, day of the week, and campus location. The target variable, Congestion Level, is classified into three categories — Low, Medium, and High. Raw data issues such as missing values, duplicates, outliers, inconsistent formatting, and erroneous entries were intentionally included to simulate realistic unprocessed data.

Phase 2 **: Data Preprocessing and Cleaning**

The raw dataset was cleaned through multiple steps. Duplicate records were identified and removed. Inconsistent categorical values such as mixed casing in Day of Week and Congestion Level columns were standardized. The Throughput column contained string-formatted values like "145.3 Mbps" which were converted to float. Negative values in the connected devices column caused by sensor errors were corrected using absolute conversion. Outliers were detected and capped using the IQR method, and missing values across eight columns were imputed using KNN Imputer.

Phase 3 **: Exploratory Data Analysis**

EDA was conducted to understand the underlying patterns and behaviour of the network data. Distribution plots were generated for all numerical features to study their spread and skewness. Congestion patterns were analysed across different hours of the day and campus locations. A correlation heatmap was plotted to identify feature relationships, and boxplots were used to compare feature distributions across congestion classes. Anomaly detection was performed using Isolation Forest to identify irregular network behaviour.

Phase 4 **: Encoding and Normalization**

Categorical features were converted into numerical form before modelling. Label Encoding was applied to the target variable mapping High to 0, Low to 1, and Medium to 2. One-Hot Encoding was applied to the Day of Week and Location columns. All numerical features were then standardized using Standard Scaler which transforms each feature to have a mean of zero and a standard deviation of one, ensuring no single feature dominates model training due to scale differences.

Phase 5 **: Feature Selection**

Three feature selection techniques were applied to identify the most relevant features for predicting congestion. Principal Component Analysis reduced the dimensionality of the feature space while retaining 95% of the variance, resulting in 16 principal components. Forward Selection iteratively added features that most improved model accuracy. Backward Elimination started with all features and progressively removed the least significant ones. The PCA-transformed feature set was selected as the final input for model training and saved as a CSV for use in the next phase.

Phase 6 **: Model Training and Evaluation**

SMOTE was applied to handle class imbalance among the three congestion categories. The dataset was split into 80% training and 20% testing sets using stratified sampling. Six classification algorithms were trained — Logistic Regression, Decision Tree, Random Forest, Gradient Boosting, SVM, and KNN. Each model was evaluated using accuracy, classification report, and confusion matrix. Hyperparameter tuning was performed on the best performing model using GridSearchCV and the final model was validated using 10-Fold Cross Validation.

Phase 7 **: Final Model and Deployment**

The Random Forest Classifier emerged as the best performing model with an accuracy of 98.20%. The trained model was saved as a pickle file using Python's pickle library. A standalone Python script, Predictor.py, was developed to load the saved model and perform predictions on new data — both in batch mode from a CSV file and as single custom inputs — making the system ready for practical deployment.

Expected Modules/ Libraries to be used

- ***<u>Data Handling & Processing</u>***

Library / Module	Description
`pandas`	Used for loading, manipulating, and processing structured CSV data throughout all notebooks
`numpy`	Used for numerical computations, array operations, and handling mathematical transformations
`os`	Used to create directories programmatically such as the models folder for saving the trained model
`pickle`	Used to serialize and save the trained Random Forest model as a .pkl file and reload it in Predictor.py

---

- ***<u>Data Preprocessing & Cleaning</u>***

Library / Module	Description
`sklearn.impute.KNNImputer`	Used to fill missing values by referencing the K nearest neighbouring data points
`sklearn.impute.SimpleImputer`	Used as a fallback imputer to fill any remaining NaN values before PCA
`sklearn.preprocessing.LabelEncoder`	Used to encode the target variable Congestion Level into numerical values 0, 1, and 2

`sklearn.preprocessing.StandardScaler`	Used to normalize all numerical features to mean zero and standard deviation one

---

- ***Exploratory Data Analysis***

Library / Module	Description
`matplotlib.pyplot`	Used to create all static plots including histograms, bar charts, and confusion matrix visualizations
`seaborn`	Used to generate heatmaps, boxplots, pairplots, and distribution plots for EDA

- ***Feature Selection***

Library / Module	Description
`sklearn.decomposition.PCA`	Used to reduce feature dimensions while retaining 95% of the total variance in the dataset
`mlxtend.feature_selection.SequentialFeatureSelector`	Used to perform both Forward Selection and Backward Elimination on the feature set

---

- ***Handling Class Imbalance***

Library / Module	Description
`imblearn.over_sampling.SMOTE`	Used to synthetically oversample the minority classes Medium and High to balance the dataset

---

- ***Model Training***

Library / Module	Description
`sklearn.linear_model.LogisticRegression`	A linear classification model used as a baseline classifier
`sklearn.tree.DecisionTreeClassifier`	A tree-based model that splits data based on feature thresholds to classify congestion
`sklearn.ensemble.RandomForestClassifier`	An ensemble of decision trees that delivered the best accuracy of 98.20%
`sklearn.ensemble.GradientBoostingClassifier`	A boosting model that builds trees sequentially to correct previous errors
`sklearn.svm.SVC`	Support Vector Machine using an RBF kernel to find the optimal decision boundary
`sklearn.neighbors.KNeighborsClassifier`	Classifies records based on the majority class among the K nearest neighbours

---

- ***<u>Model Evaluation & Tuning</u>***

Library / Module	Description
`sklearn.model_selection.train_test_split`	Used to split the dataset into 80% training and 20% testing sets
`sklearn.model_selection.GridSearchCV`	Used to exhaustively search the best hyperparameters for the Random Forest model
`sklearn.model_selection.cross_val_score`	Used to perform 10-Fold Cross Validation to assess model generalization
`sklearn.metrics.accuracy_score`	Used to calculate the overall prediction accuracy of each trained model
`sklearn.metrics.classification_report`	Used to generate precision, recall, and F1-score for each congestion class
`sklearn.metrics.confusion_matrix`	Used to visualize correct and incorrect predictions across all three classes
`sklearn.metrics.ConfusionMatrixDisplay`	Used to render the confusion matrix as a readable colour-coded plot

---

- ***Anomaly Detection***

Library / Module	Description
`sklearn.ensemble.IsolationForest`	Used during EDA to detect anomalous network behaviour records in the dataset

Expected Evaluation metrics and Plots to be used

- ***<u>Evaluation Metrics</u>***

1. Accuracy Score

Accuracy is the primary evaluation metric used in this project as explicitly required by the problem statement. It measures the percentage of correctly classified records out of the total predictions made by the model. It gives a quick and straightforward overview of how well the model is performing overall across all three congestion classes — Low, Medium, and High. After applying SMOTE to balance the classes, accuracy becomes a reliable and fair metric for this project.

2. Classification Report (Precision, Recall, F1-Score)

Since this is a multi-class classification problem with three congestion levels, accuracy alone does not reveal how well the model performs on each individual class. Precision measures how many of the predicted congestion cases were actually correct. Recall measures how many actual congestion cases the model successfully identified. F1-Score is the harmonic mean of precision and recall and is particularly useful when evaluating performance on minority classes like High congestion which are critical to detect correctly in a real network management scenario.

3. Confusion Matrix

The confusion matrix provides a detailed breakdown of correct and incorrect predictions for each class. It shows exactly how many Low records were predicted as Medium or High and vice versa. This helps identify which congestion class the model struggles with the most and gives deeper insight beyond what a single accuracy number can provide.

4. Cross Validation Score (10-Fold)

Cross validation is used to assess whether the model generalizes well to unseen data or is simply overfitting the training set. By splitting the data into 10 folds and training and testing across each fold, the mean accuracy and standard deviation are reported. A low

standard deviation confirms that the model performs consistently and is not dependent on any particular train-test split.

- ***Visualization Plots***

1. Distribution Plots (Histogram with KDE)

Used during EDA to understand the spread, skewness, and shape of each numerical feature such as latency, bandwidth, and packet loss. This helps identify whether features are normally distributed or heavily skewed before applying scaling.

2. Correlation Heatmap

Used to visualize the pairwise correlation between all numerical features. It reveals which features are strongly related to each other and to the target variable, helping guide feature selection decisions. Highly correlated features can be candidates for removal to reduce redundancy.

3. Boxplots by Congestion Class

Used to compare the distribution of each feature across Low, Medium, and High congestion levels. This directly shows which features have the most discriminating power between classes — for example whether high AP Load consistently corresponds to High congestion.

4. Pairplot

Used to visualize relationships between multiple features simultaneously, color-coded by congestion class. It helps identify linear and non-linear separability between the three classes and reveals which feature combinations are most useful for classification.

5. Congestion by Hour of Day (Line / Box Plot)

Used to identify peak congestion periods throughout the day. This visualization directly addresses the requirement of identifying congestion periods and understanding time-based network behaviour patterns across the campus.

6. Congestion by Location (Count Plot)

Used to identify which campus zones such as Library, Lab Blocks, and Cafeteria experience the highest congestion frequency. This gives location-specific insights that support real-world network management decisions.

7. PCA Explained Variance Plot (Bar Chart)

Used during feature selection to visualize how many principal components are needed to retain 95% of the total variance. The cumulative explained variance curve helps justify the number of components selected and validate the dimensionality reduction step.

8. Model Comparison Bar Chart

Used at the end of Notebook 4 to visually compare the accuracy of all six trained classifiers side by side. This makes it immediately clear which model performed best and by how much, supporting the final model selection decision.

9. Confusion Matrix Heatmap

Used as the final evaluation visualization for the best performing model. It is rendered as a color-coded grid showing true positives, false positives, and false negatives for each congestion class, making it easy to interpret prediction errors at a glance.

Required topics from the subject

1. Variables and Data Types

Understanding different data types such as integers, floats, strings, and booleans is fundamental to this project. Network features like latency, packet loss, and bandwidth are stored and manipulated as numerical data types throughout the pipeline. Correct data type handling was also a key cleaning step where Throughput values stored as strings were converted back to float.

---

2. Lists and Arrays

Lists and NumPy arrays are used extensively to store feature values, prediction outputs, and transformed datasets. Operations like slicing, indexing, and iterating over arrays are performed during preprocessing, feature selection, and model evaluation. Understanding how arrays behave under mathematical operations is essential for scaling and PCA transformations.

---

3. Functions and Modules

The entire project is built using Python functions and imported modules from libraries like scikit-learn, pandas, and seaborn. Understanding how to define functions, pass arguments, and return values is necessary to structure clean and reusable code across all four notebooks. Importing and using external modules correctly is a core skill used in every phase of the pipeline.

---

4. File Handling

Reading and writing CSV files using pandas is central to this project as data is passed between notebooks through saved CSV files. The trained model is saved and loaded using Python's pickle module which is a form of binary file handling. Understanding file

paths, file modes such as read and write, and directory creation using the os module is directly applied in this project.

---

5. Conditional Statements and Loops

Conditional statements are used to define congestion labels based on threshold values of AP load, latency, and packet loss during dataset generation. Loops are used to iterate over multiple models during training, evaluation, and comparison. Understanding how to use if-else logic and for loops efficiently is essential for automating repetitive tasks across the ML pipeline.

---

6. String Operations

String manipulation was a direct requirement in the data cleaning phase where values like "145.3 Mbps" needed to have the " Mbps" suffix stripped before being converted to float. Inconsistent categorical values with mixed casing such as "monday", "FRIDAY", and "HIGH" were corrected using string methods like strip, title, and replace. Understanding basic string operations is therefore a practical necessity in real-world data preprocessing.

---

7. Object Oriented Programming (OOP)

All scikit-learn models and preprocessing tools such as StandardScaler, LabelEncoder, and RandomForestClassifier are implemented as classes and objects. Understanding OOP concepts like instantiation, methods, and attributes is necessary to correctly initialize, fit, and use these tools. The concept of fitting an object on training data and transforming test data separately is a direct application of OOP principles.

---

8. Exception Handling

Try-except blocks are used in the prediction script to gracefully handle cases where a model may not support probability output. Exception handling ensures the program does

not crash unexpectedly during execution and instead provides a meaningful fallback message. Writing robust code with proper error handling is an important software practice especially when deploying models in a standalone script.

---

9. Data Structures — Dictionary and DataFrame

Python dictionaries are used to store and iterate over multiple trained models simultaneously during the training and evaluation phase. Pandas DataFrames are the primary data structure used throughout the project for loading, cleaning, transforming, and analysing the dataset. Understanding how to access, filter, group, and manipulate rows and columns in a DataFrame is a core skill applied in every notebook.

---

10. Statistics and Probability Fundamentals

Concepts like mean, standard deviation, variance, and distribution are directly applied during EDA and normalization. Understanding correlation coefficients is necessary to interpret the heatmap and make informed feature selection decisions. Basic probability concepts underpin how models like Logistic Regression and Random Forest generate class probabilities during prediction.

Platform Required

The following platform and tools were used to develop and execute this end-to-end machine learning project.

Visual Studio Code (VS Code)

Visual Studio Code served as the primary development platform for this entire project. It provides an integrated environment that supports multiple file types and extensions, making it ideal for managing all components of the project within a single workspace.

The project consists of four types of files, all developed and managed within VS Code:

- **Jupyter Notebook files (.ipynb)** — Four notebooks were created within VS Code using the Jupyter extension. These notebooks were used for data cleaning, exploratory data analysis, feature selection, and model training and evaluation respectively. The notebook format allowed code, outputs, and visualizations to be viewed and executed interactively in the same environment.
- **Python Script file (.py)** — The standalone Predictor.py file was written and executed directly within VS Code's integrated terminal. This script loads the saved model and performs congestion predictions without requiring Jupyter.
- **CSV files (.csv)** — All dataset files including the raw dataset, cleaned dataset, encoded and normalised dataset, and feature selected dataset were stored and accessed as CSV files within the project folder. These were read and written using pandas throughout the pipeline.
- **Model file (.pkl)** — The final trained Random Forest model was saved as a pickle file inside the models folder within the project directory. This binary file was generated through VS Code and is reloaded by the Predictor.py script for deployment.

Books and Link Sources

Books

Aurélien Géron — *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3rd Edition, O'Reilly Media, 2022) A comprehensive practical guide covering the entire machine learning pipeline including data preprocessing, feature selection, classification algorithms, and model evaluation using scikit-learn — directly relevant to every phase of this project.

Sebastian Raschka & Vahid Mirjalili — *Python Machine Learning* (3rd Edition, Packt Publishing, 2019) Covers classification algorithms, dimensionality reduction using PCA, and model evaluation techniques in Python in depth. Particularly useful for understanding the theoretical foundations behind Random Forest, SVM, and Logistic Regression used in this project.

Online Documentation References

Scikit-learn Official Documentation All machine learning models, preprocessing tools, feature selection methods, and evaluation metrics used in this project are documented here. 🔗 https://scikit-learn.org/stable/documentation.html

Imbalanced-learn Official Documentation Official reference for SMOTE and other oversampling techniques used to handle class imbalance in this project. 🔗 https://imbalanced-learn.org/stable/

Seaborn Official Documentation Reference for all statistical visualization plots used during the EDA phase including heatmaps, boxplots, and distribution plots. 🔗 https://seaborn.pydata.org/

Research Paper

Breiman, L. — *Random Forests*, Machine Learning Journal, Vol. 45, pp. 5–32, Springer, 2001 The original research paper by Leo Breiman introducing the Random Forest algorithm — the best performing model in this project with 98.20% accuracy. Useful for citing the theoretical foundation of the algorithm. 🔗 https://link.springer.com/article/10.1023/A:1010933404324